Task 03: Network Security Monitoring and Analysis

Cyber Security Intern – SOC Domain
Company: CyArt
Prepared by:
Sudeep
Tools Used:

- Wireshark
- Snort IDS
- Scapy
- Kali Linux

1. Introduction

This task focuses on **network security monitoring and analysis using Wireshark, Snort, and Scapy**.
The goal of this exercise is to understand how network traffic flows in a system, how to detect suspicious activity using intrusion detection systems, and how to generate custom network packets for testing security rules.
Through this task, we analyzed network traffic using Wireshark, configured Snort as an intrusion detection system, and used Scapy to simulate network attacks such as SYN scans.

2. Tools Used

Tool	Purpose
Wireshark	Packet capture and traffic analysis
Snort	Intrusion Detection System
Scapy	Packet generation and network testing
Kali Linux	Security testing environment

3. Network Interface Identification

The network interface used for monitoring traffic was identified using the command:

ip a

The active interface used for monitoring was:
eth0

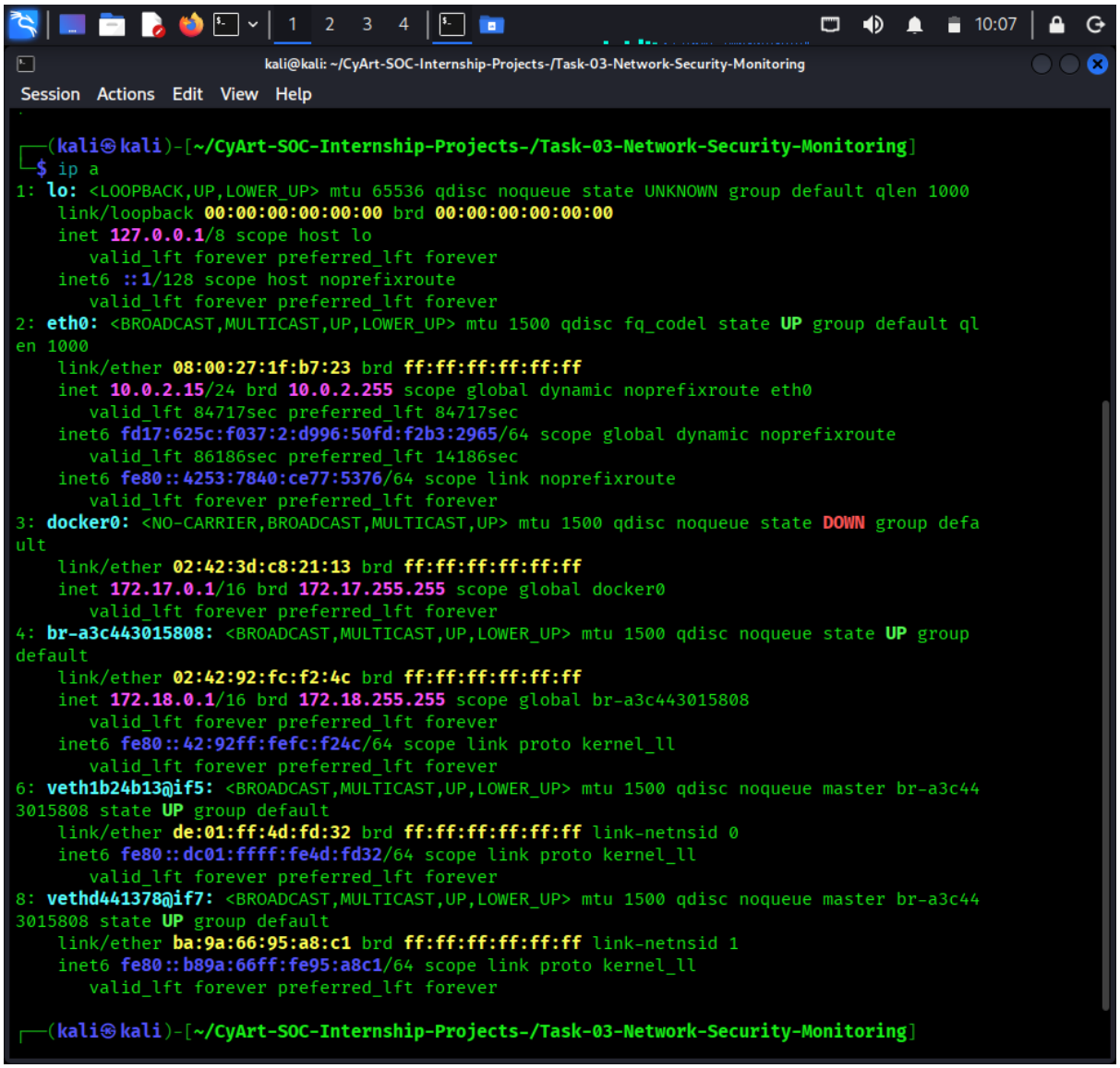


Figure 1: Network Interface Identification using the 'ip a' Command (eth0)

4. Wireshark Traffic Analysis

Wireshark was used to capture and analyze network traffic for approximately 10 minutes
During this period, multiple types of network protocols were observed including:

- TCP
- UDP
- ICMP

These protocols represent common network communications such as web browsing, DNS queries, and ping requests.

TCP Traffic

Filter used:

tcp

This filter shows all TCP communication such as HTTP requests and SSH connections.

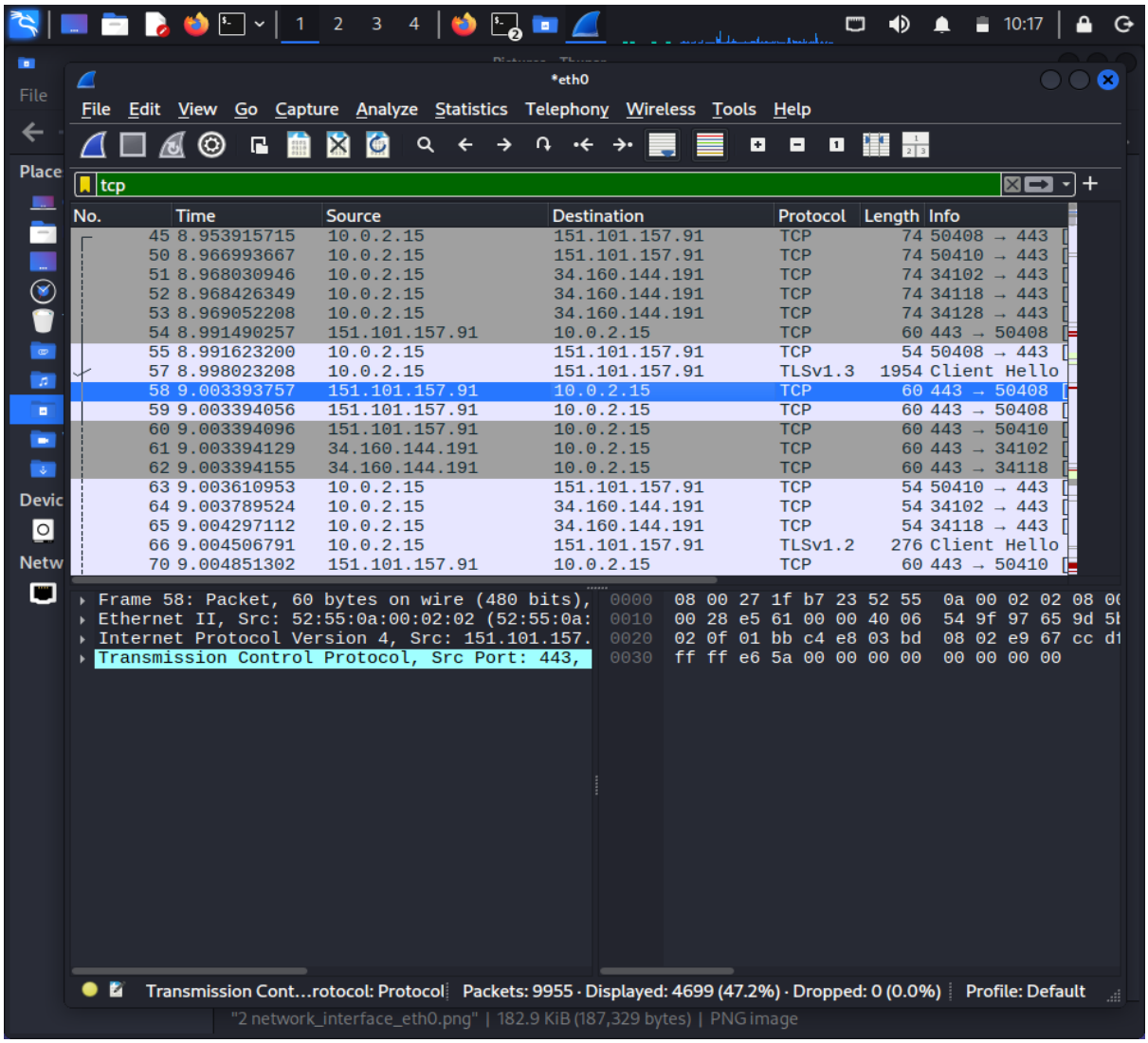


Figure 2: Wireshark TCP Protocol Traffic Filtering

UDP Traffic

Filter used:

udp

UDP traffic includes DNS queries and other fast network communications.

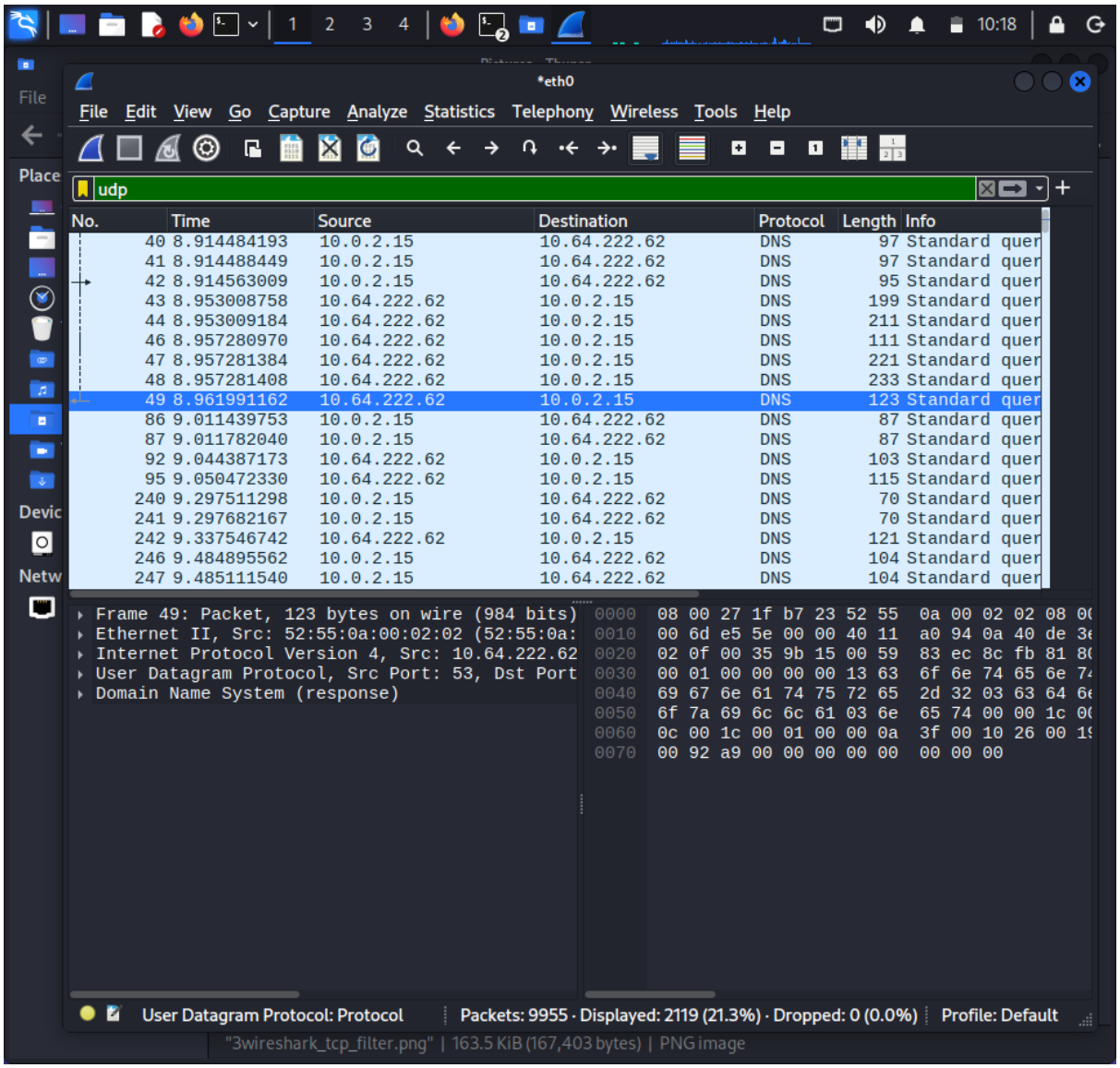


Figure 3: Wireshark UDP Protocol Traffic Filtering

ICMP Traffic

Filter used:
icmp
ICMP packets are commonly used for diagnostic tools like **ping**.

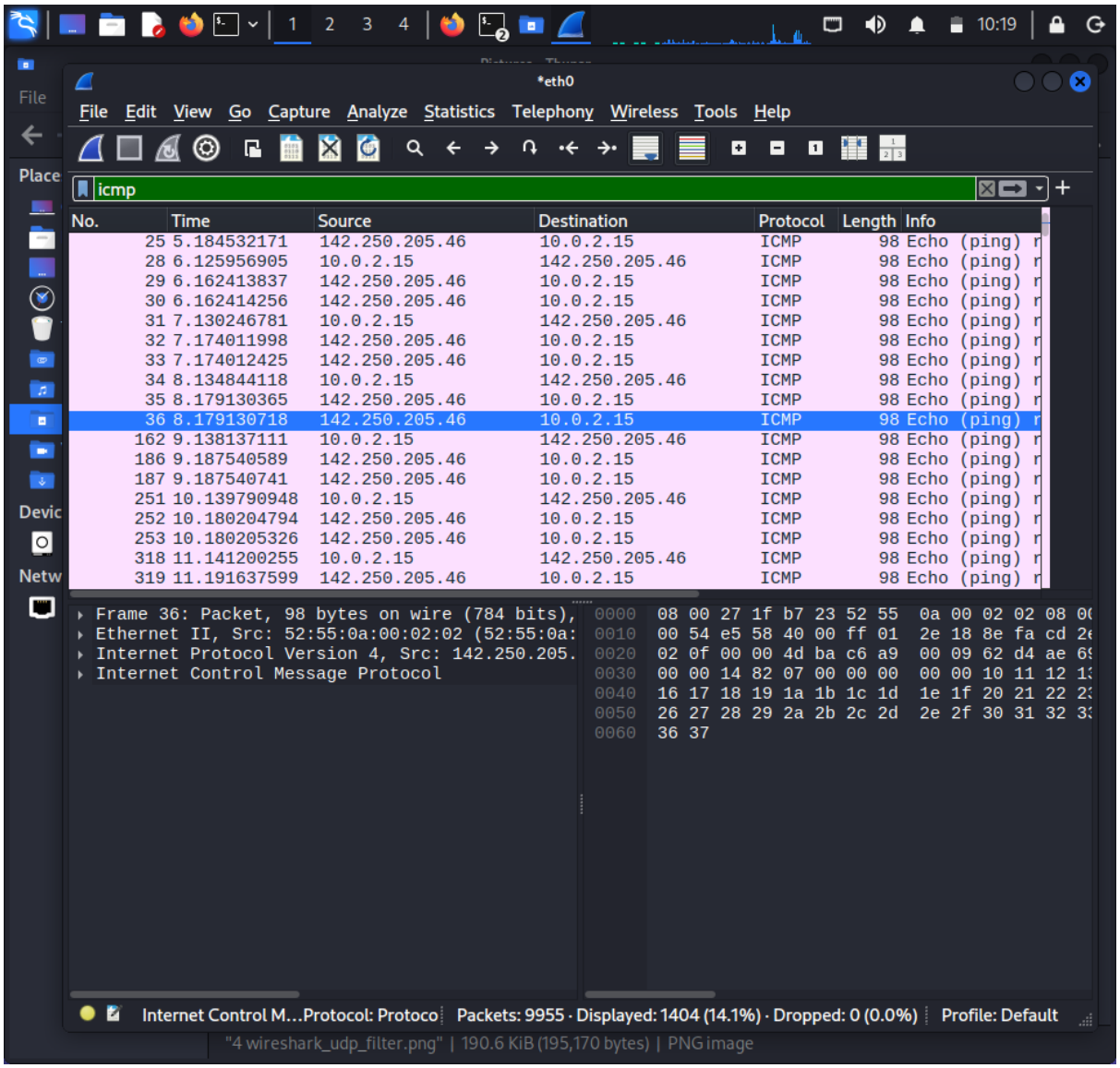


Figure 4: Wireshark ICMP Packet Analysis

Protocol Distribution Analysis

During the traffic capture period, multiple network protocols were observed including TCP, UDP, and ICMP. TCP traffic represented the majority of the captured packets due to common web communication such as HTTP and HTTPS requests. UDP traffic was primarily associated with DNS queries, while ICMP packets were generated from network diagnostic commands such as ping. This distribution reflects normal network activity in a local environment.

SYN Packet Detection

To detect SYN packets the following filter was used:
tcp.flags.syn == 1 && tcp.flags.ack == 0
SYN packets are used in the **TCP handshake process** and may indicate scanning activity if repeated.

Security Observation

Repeated SYN packets without corresponding ACK responses may indicate reconnaissance activity such as SYN scanning. Attackers use this technique to identify open ports on a target system before launching further attacks.

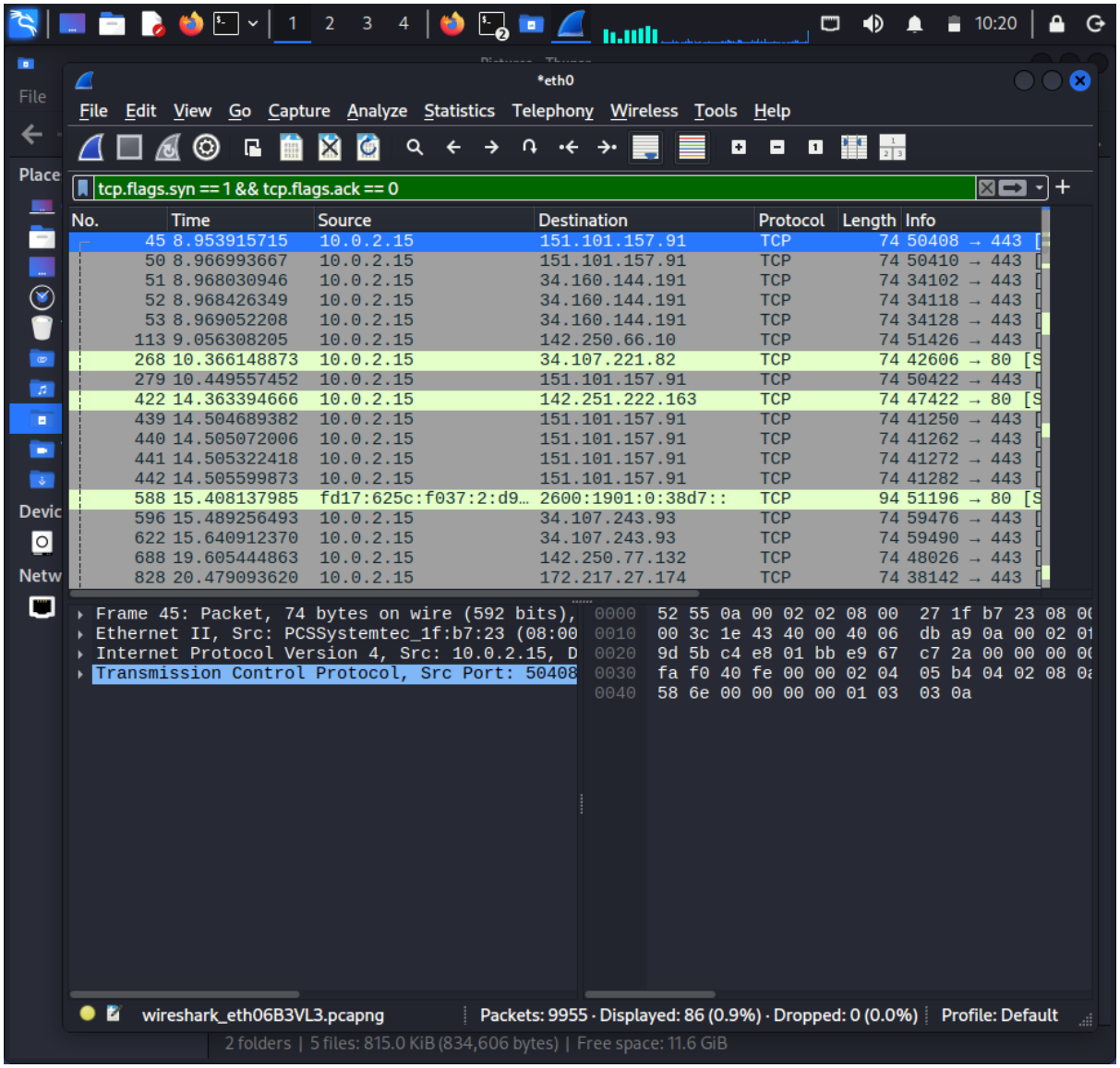


Figure 5: Detection of TCP SYN Packets Using Wireshark Filter

5. Snort IDS Configuration

Snort was configured as a **Network Intrusion Detection System (IDS)** to detect suspicious traffic.

Custom rules were added to detect:

- SYN packets

- Port scanning activity

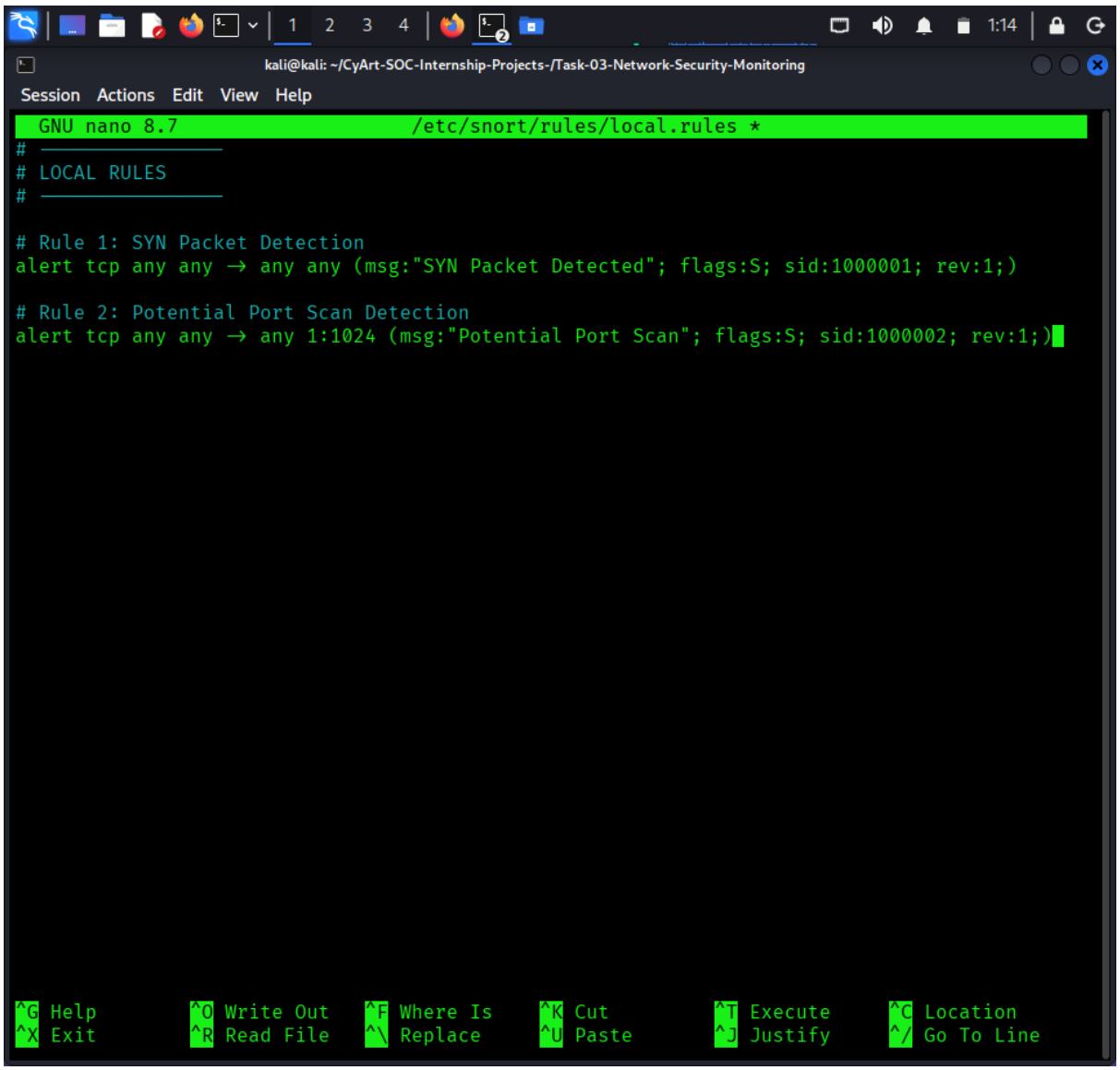
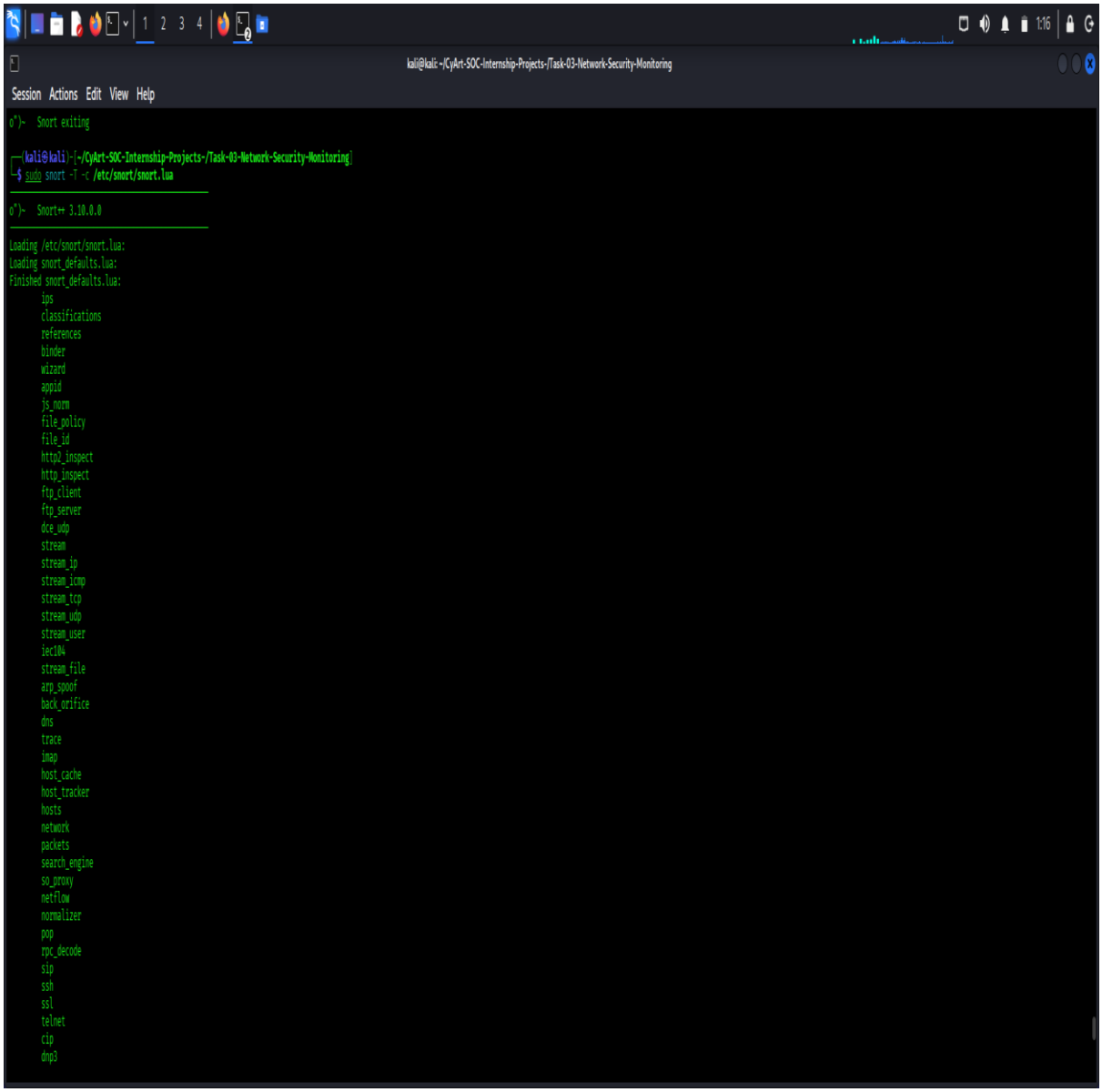


Figure 6: Custom Snort Rules Configuration for SYN Packet and Port Scan Detection

6. Running Snort IDS

Snort was executed using the following command:
sudo snort -i eth0 -c /etc/snort/snort.lua -A console

This command starts Snort in IDS mode and displays alerts in the console.



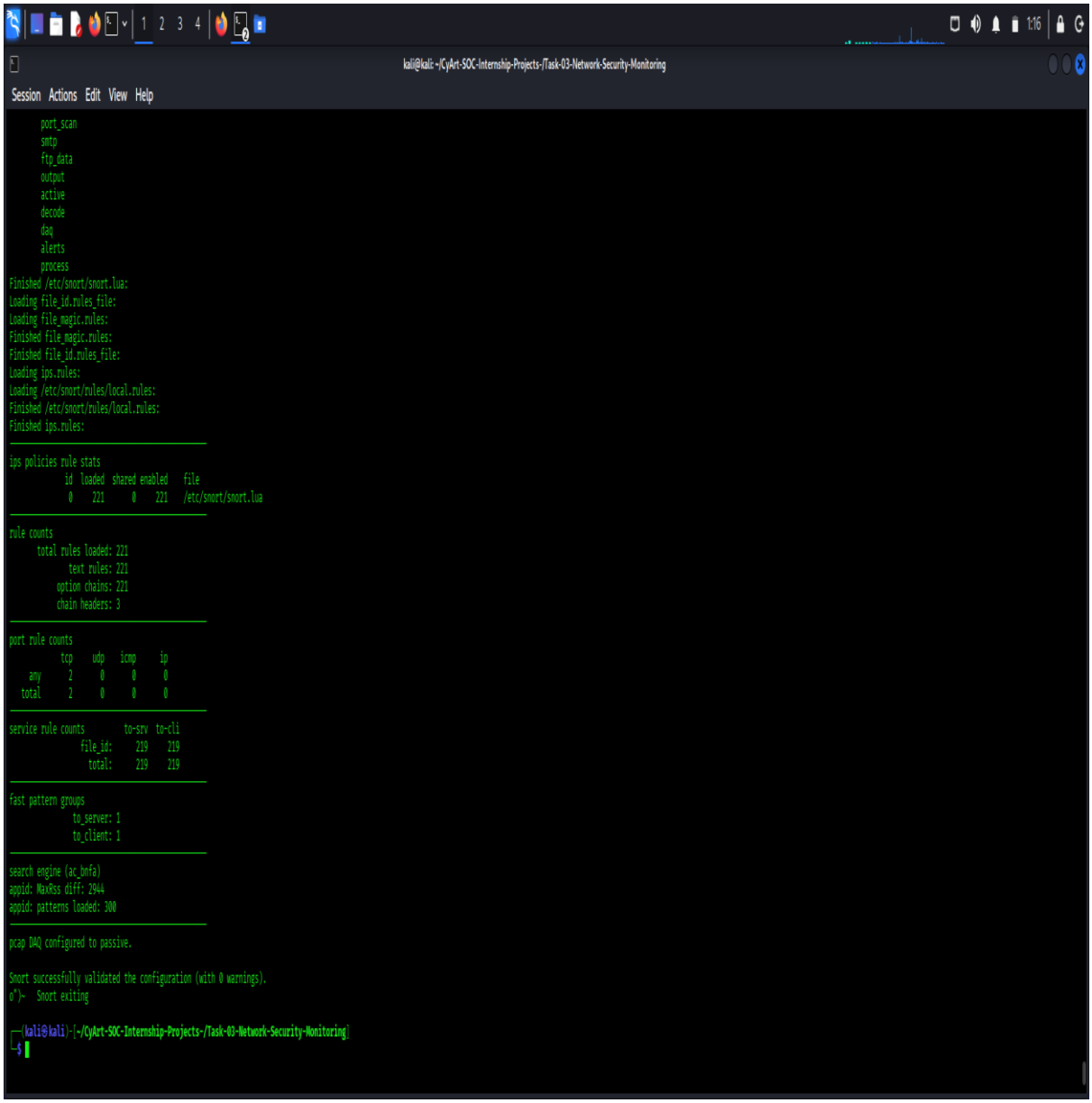


Figure 7 & 8: Snort Intrusion Detection System Running in Console Mode

Detection Analysis

When the Scapy script generated SYN packets targeting multiple ports, Snort successfully triggered the custom IDS rules. The alerts indicate potential port scanning behavior, which is commonly used by attackers during the reconnaissance phase of an attack

7. Generating Test Traffic with Scapy

A Python script was created using Scapy to simulate **SYN packet scans**.
This script sends SYN packets to multiple ports to simulate a **port scan attack**.
Script name:
test_packets.py

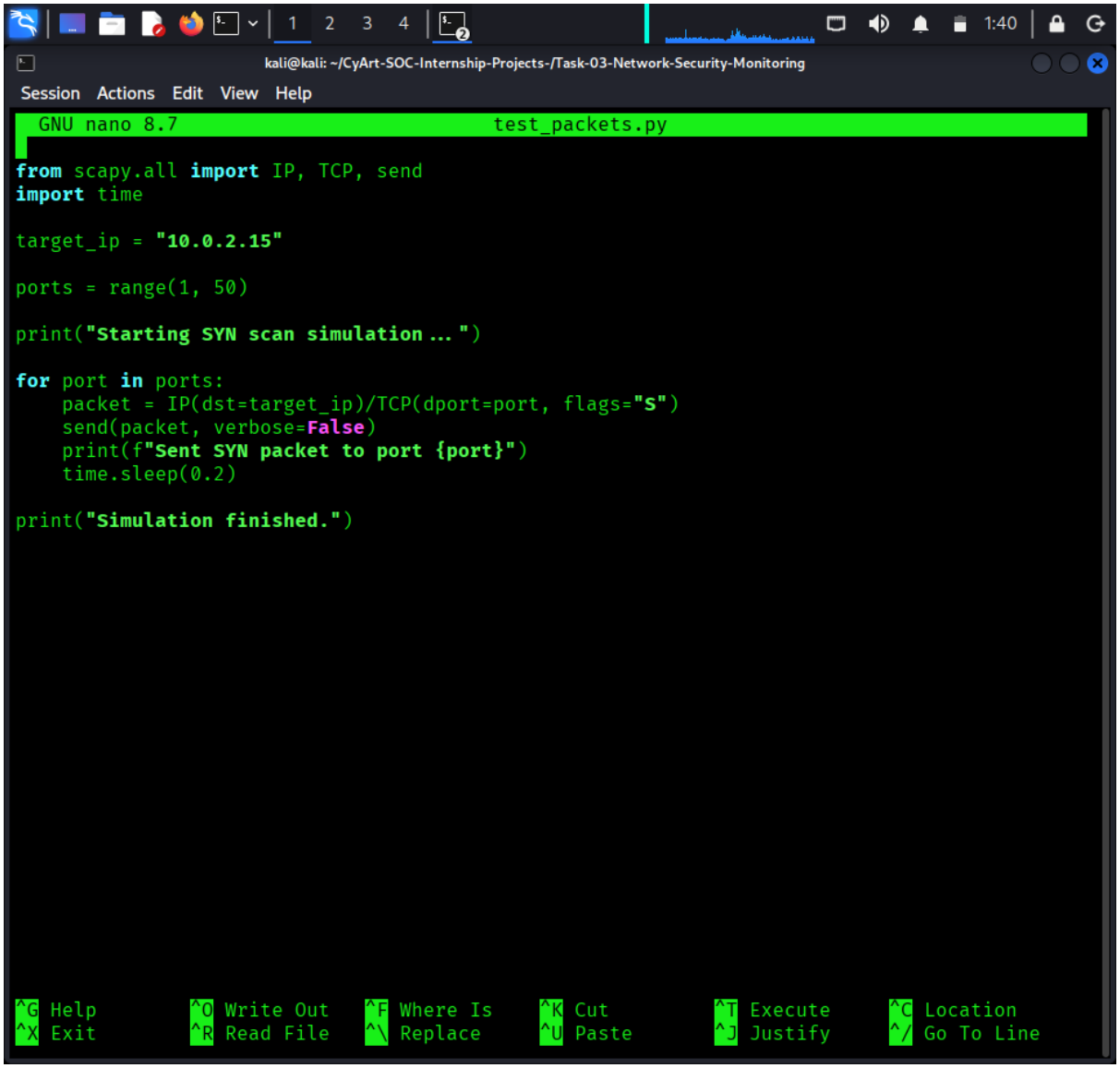


Figure 9: Python Scapy Script Used for Generating SYN Scan Traffic

8. Snort Detection Results

After running the Scapy script, Snort successfully detected the generated traffic and produced alerts.

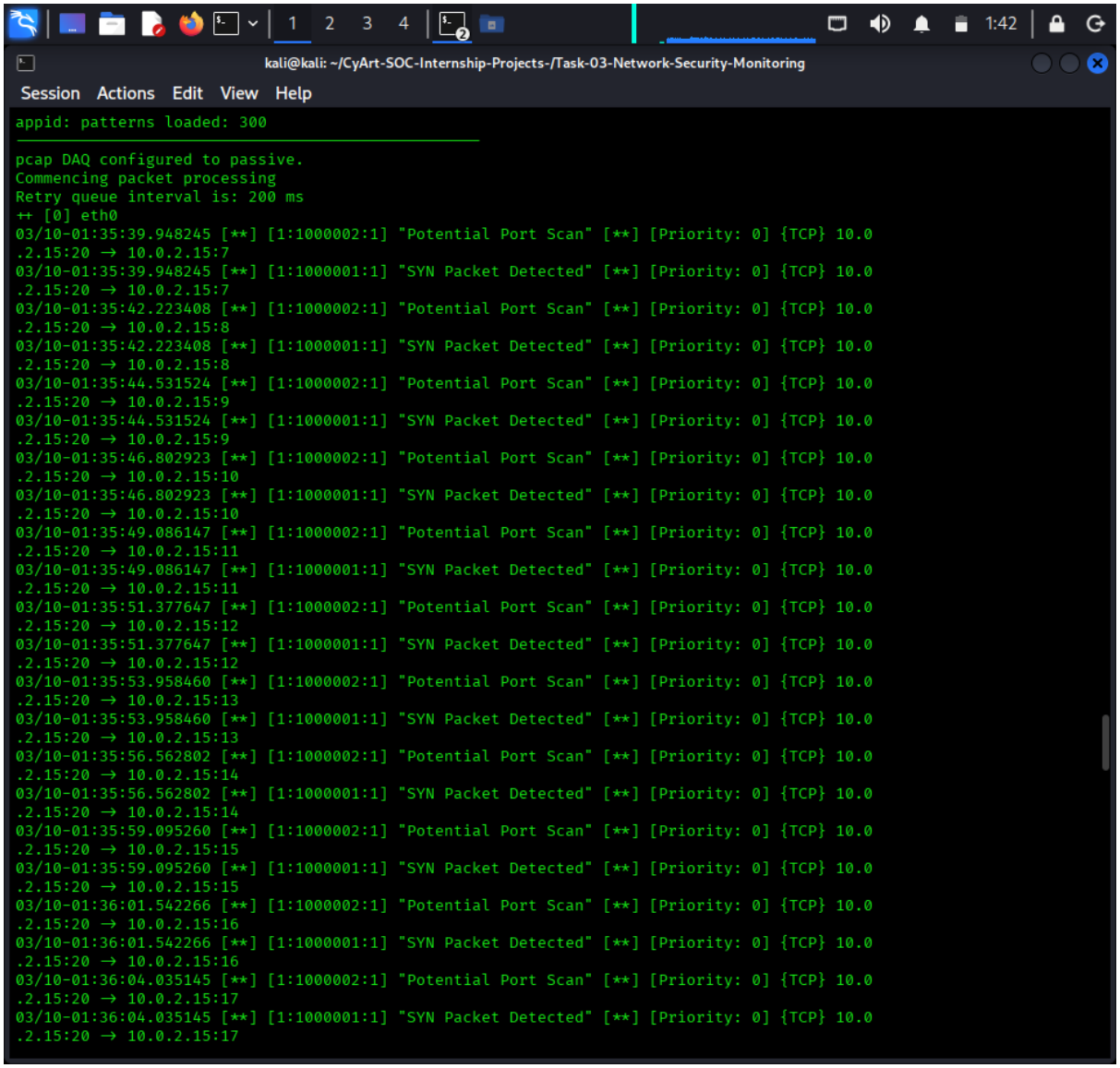


Figure 10: Snort Alert Output Detecting SYN Packets and Port Scan Activity

9. Key Learnings

Through this task, I gained practical knowledge in:

- Network traffic analysis using Wireshark
- Creating and testing IDS rules using Snort
- Generating custom packets using Scapy
- Identifying suspicious traffic patterns such as SYN scans

This exercise improved my understanding of **network monitoring and intrusion detection techniques used by SOC analysts**.

10. Challenges Faced

Some challenges faced during this task included:

- Configuring Snort rules correctly
- Generating test traffic that triggers IDS alerts
- Filtering specific packet types in Wireshark

These issues were resolved by reviewing official documentation and testing multiple configurations.

11. Conclusion

This task provided hands-on experience in network security monitoring and intrusion detection.

By combining Wireshark, Snort, and Scapy, it was possible to analyze network traffic, simulate attacks, and detect malicious activity effectively.

This practical knowledge is essential for **SOC analysts responsible for monitoring and defending networks against cyber threats**.